

使用稀疏 Transformer 生成序列

雷翁·蔡尔德¹ 斯科特·格雷¹ 亚历克·拉德福德¹ 伊利亚·萨茨凯弗¹

摘要

Transformer 是强大的序列模型，但所需的时间和内存会随着序列长度的增加呈二次方增长。在本文中，我们提出了注意力矩阵的稀疏分解方法，将其降至 $O(n\sqrt{n})$ 。我们还提出了：a) 一种架构和初始化的变体，用于训练更深层次的网络；b) 重新计算注意力矩阵以节省内存；c) 用于训练的快速注意力核。我们将经过这些改进的网络称为稀疏 Transformer，并证明它们能够使用数百层来建模长达数万个时间步的序列。我们使用相同的架构从原始字节中对图像、音频和文本进行建模，在 Enwik8、CIFAR10 和 ImageNet-64 的密度建模方面创下了新的最先进水平。我们生成的无条件样本展示了全局一致性和高度多样性，并证明原则上可以使用自注意力来建模长度为一百万或更长的序列。

· 引言

估计复杂的高维数据分布是无监督学习中的一个核心问题，因为许多令人关注的下游应用都涉及文本、图像、音频和其他数据的生成。此外，它还被认为是无监督表示学习的一个关键组成部分。

最近，神经自回归模型在该领域取得了令人瞩目的成果，在自然语言建模 (Jozefowicz 等人, 2016) (Radford 等人, 2018) (Dai 等人, 2018)、原始音频 (Van Den Oord 等人, 2016) (Mehri 等人, 2016) 以及图像 (Oord 等人, 2016) (Menick 和 Kalchbrenner, 2018) (Salimans 等人, 2017) (Reed 等人, 2017) (Chen 等人, 2017) 方面达到了最先进水平。

这些方法将联合概率分布分解为条件概率分布的乘积。然而，对这些条件分布进行建模极具挑战性，因为它们包含许多复杂的长程依赖关系，并且需要适当的具有表达能力的模型架构来对其进行学习。

基于 CNN 的架构 (Oord 等人, 2016) 已经取得了

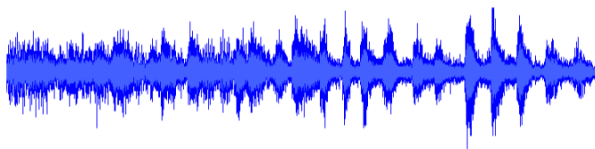
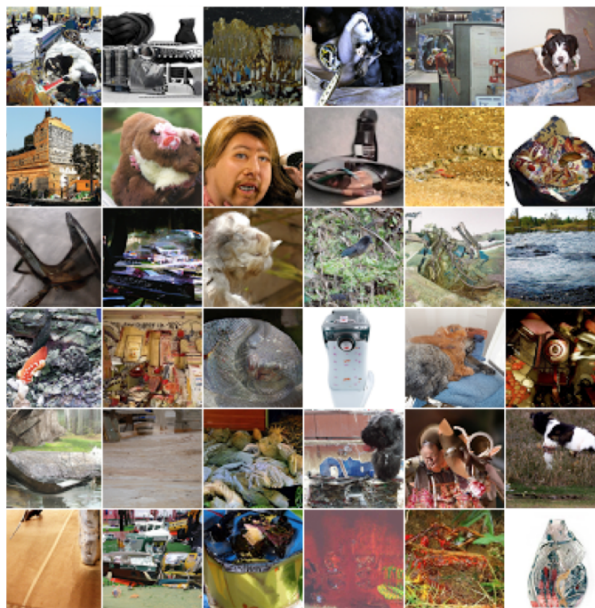


图 1. 我们的神经自回归模型在 ImageNet 64 和一个古典音乐数据集上的无条件样本。我们对音频、图像和文本使用了相同的基于自注意力的架构。上面的样本是在 softmax 温度为 1.0 的情况下生成的，长度分别为 12,288 和 65,536。可在

<https://openai.com/blog/sparse-transformer> 收听音频样本。

在这个方向上已经取得了重大进展，但需要足够的深度来扩大其感受野。为了解决这个问题，WaveNet (Van Den Oord 等人, 2016) 引入了膨胀卷积，这使得网络能够在对数数量的层中对长程依赖关系进行建模。

此外，Transformer (Vaswani 等人, 2017) 已被证明在许多自然语言任务上表现出色，这在一定程度上可能是由于它能够在固定数量的层中对任意依赖关系进行建模。由于每个自注意力层都具有全局感受野，网络可以将表征能力分配给输入区域，对于这些区域，它

因此，与具有固定连接模式的网络相比，这种架构在生成多样化数据类型方面可能更具灵活性。

然而，此类网络的内存和计算需求会随着序列长度的增加呈二次方增长，这使其无法用于长序列。

这项工作的主要贡献是引入了注意力矩阵的几种稀疏分解方法，这些方法在不牺牲性能的情况下，与序列长度的缩放关系为 $\sqrt{O(n \sqrt{n})}$ 。它们的工作原理是将完整的注意力计算分解为几个更快的注意力操作，这些操作结合起来可以近似密集注意力操作。我们利用这一点将自注意力应用于前所未有的长序列。

此外，我们还对 Transformer 引入了其他几项改动。包括：

- 一种重构的残差块和权重初始化方法，以改进极深网络的训练
- 一组稀疏注意力内核，可高效计算注意力矩阵的子集
- 在反向传播过程中重新计算注意力权重以减少内存使用量

我们通过实证验证，以这种方式增强的模型能够在自然语言、原始音频和自然图像的压缩与生成方面达到最先进水平。该架构的简洁性使我们相信，它可能对许多重要问题有用。

· 相关工作

最相关的研究工作涉及其他用于扩展自回归生成模型的技术。在图像方面，(Reed 等人, 2017) 对像素之间的条件独立性进行建模，以便并行生成多个位置；(Menick 和 Kalchbrenner, 2018) 采用一种排序和多尺度上采样流程来生成高保真样本。(Parmar 等人, 2018) 使用局部注意力块将 Transformer 应用于图像。在文本方面，(Dai 等人, 2018) 引入了一种状态重用“记忆”来建模长期依赖关系。在音频方面，除了 (Van Den Oord 等人, 2016) 的研究外，(Mehri 等人, 2016) 使用分层结构和不同时钟频率的循环神经网络 (RNN)，在推理过程中利用长上下文，这与 (Koutnik 等人, 2014) 的研究类似。(Huang 等人, 2018) 通过高效的相对注意力将 Transformer 应用于 MIDI 生成。

我们的工作比上述许多技术更简单，并且可以同等地应用于图像、文本和音频。此外，上述许多技术与我们的技术互不冲突，还可以与我们的技术结合使用。

在生成建模之外，有几项研究与基于分块 (Chiu & Raffel, 2017) 或使用固定长度表示 (Britz 等人, 2017) 来提高注意力机制的效率相关。其他研究则对具有多个“跳跃”的注意力机制进行了探究，例如 (Sukhbaatar 等人, 2015) 和 (Gehring 等人, 2017) 的研究。

值得注意的是，门控像素 CNN (Oord 等人, 2016) 和 WaveNet (Van Den Oord 等人, 2016) 在其网络中使用了乘法交互。这与自注意力有关。

· 背景

我们考虑自回归序列生成任务，其中序列 $x =$

$x_1, x_2, \dots, x_n$ 的联合概率被建模为条件概率分布的乘积，并由网络 θ 参数化。

$$p(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1}; \theta)$$

我们将图像、文本和音频视为离散标记的序列，通常是原始字节。网络 θ 接收标记序列，并使用 softmax 函数输出下一个标记在 U 种可能值上的分类分布，其中 v 是词汇表的大小。训练目标是相对于 θ 最大化数据的对数概率。

对于模型 θ 来说，一个简单而强大的选择是仅解码器模式的 Transformer (Vaswani 等人, 2017)，这一点在 (Radford 等人, 2018) 和 (Liu 等人, 2018) 的研究中得到了证明。这些模型通过对整个序列进行多头自注意力块处理，然后对每个序列元素进行密集变换，来转换输入序列。然而，网络的自注意力部分必须为 n 个元素中的每一个计算 n 个权重，随着序列长度的增加，这很快会变得难以处理。

在接下来的章节中，我们将介绍对 Transformer 架构所做的修改，这些修改使其更适合对长序列进行建模。

· 因式分解自注意力

稀疏 Transformer 将完整的自注意力操作分散到多个注意力步骤中，如图 3 (b) 和 3 (c) 所示。为了论证我们的方法，我们首先对标准 Transformer 在图像数据集上学习到的注意力模式进行了定性评估。



图 2. 在 CIFAR-10 上用全注意力训练的 128 层网络的习得注意力模式。白色高亮部分表示生成特定像素时一个注意力头的注意力权重，黑色部分表示自回归掩码。各层能够学习多种专门的稀疏结构，这或许能解释它们适应不同领域的能力。a) 网络中的许多早期层学习局部连接模式，类似卷积。b) 在第 19 层和第 20 层，网络学会将注意力分配到行注意力和列注意力上，有效地分解了全局注意力计算。c) 多个注意力层呈现出全局的、依赖数据的访问模式。d) 第 64-128 层中的典型层表现出高度稀疏性，其位置很少被激活，且仅针对特定的输入模式。

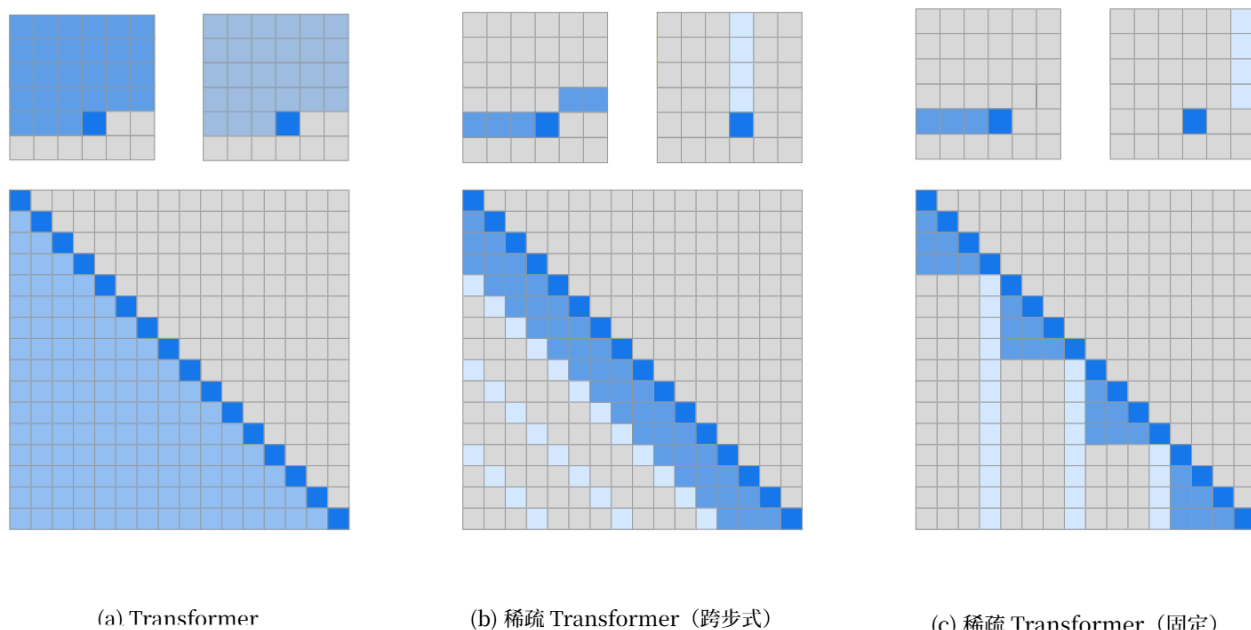


图 3. 我们评估的两种二维因式分解注意力机制，与标准 Transformer 的完全注意力机制 (a) 进行了对比。上行展示了一个 6x6 的示例图像中，两个注意力头在计算特定输出时接收的位置信息。下行展示了所有此类输出（行）和输入（列）之间的连接矩阵（未按比例绘制）。连接矩阵中的稀疏性可显著加快计算速度。在 (b) 和 (c) 中，当两个注意力头按顺序计算时，元素之间的完全连接得以保留。我们测试了这种因式分解是否能在性能上与图 2 中丰富的连接模式相匹配。

4.1. 习得注意力模式的定性评估

我们可视化了一个 128 层自注意力网络在 CIFAR-10 上学习到的注意力模式，并在图 2 中展示了几个示例。视觉检查显示，大多数层在大多数数据点上都具有稀疏的注意力模式，这表明可以引入某种形式的稀疏性，而不会显著影响性能。然而，有几层（图 2c）明显表现出全局模式，其他层则表现出数据依赖的稀疏性（图 2d），这两种情况都会因在所有注意力矩阵中引入预定的稀疏模式而受到影响。

在本文中，我们将研究范围限定在一类稀疏注意力模式上，这类模式在多个注意力步骤中所有位置之间都存在连接。这些方法可能比全注意力更高效，同时仍能为任何给定位置提供全局上下文。鉴于这些因子化模式无法学习到与图 2 中所示完全相同的映射，我们旨在通过实证验证它们在一系列任务上的性能。下面我们将介绍因子化注意力的公式。

4.2. 因式分解自注意力

自注意力层将输入嵌入矩阵 X 映射到输出矩阵，并由连接模式 $S = S_1, \dots, S_n$ 进行参数化，其中 S_i 表示第 i 个输出向量所关注的输入向量的索引集。输出向量是输入向量变换的加权和：

$$\text{Attend}(X, S) = (a(x_i, S_i))_{i \in [1 \dots n]} \quad (2)$$

$$a(x_i, S_i) = \text{softmax} \left(\frac{(W_q x_i) K S_i^T}{\sqrt{d}} \right) V S_i \quad (3)$$

$$K S_i = (W_k x_i)_{i \in \mathcal{C}}; V S_i = (W_v x_i)_{i \in \mathcal{C}} \quad (4)$$

这里， W_q 、 W_k 和 W_v 表示权重矩阵，这些矩阵将给定的 x_i 转换为查询、键或值， d 是查询和键的内部维度。每个位置的输出是值的总和，该总和由键和查询的缩放点积相似度加权得到。

自回归模型的完全自注意力机制定义了 $S_i = \{j : j < i\}$ ，使每个元素都能关注所有先前的位置及其自身位置。

相反，因式分解自注意力有 p 个独立的注意力头，其中第 m 个头定义了索引 $\Delta^{(m)} \subset \{j : j < i\}$ 的一个子集，并让 $\mathcal{C}_m = \Delta^{(m)}$ 。我们主要关注子集 A 的有效选择，其中 $|\Delta^{(m)}| \propto \sqrt{n}$ 。

此外，目前我们认为 A 的有效选择是，在注意力的 p 个步骤中，所有输入位置都与所有未来的输出位置相连。

对于每一对 $i < j$ ，我们设置每一个 A ，使得 $i \rightarrow j$ 能够通过一条位置路径关注到 j ，该路径的最大长度为 $\sqrt{n} + 1$ 。具体来说，如果 $i \rightarrow k \rightarrow \dots \rightarrow j$ 是索引路径，那么就是 $i \in A^{(1)}$ 、 $k \in A^{(2)}$ 、 $j \in A^{(3)}$ 等等。

这两个标准使我们能够保持 Transformer 在恒定步数内将信号从任意输入位置传播到任意输出位置的能力，同时将总有效计算量减少到 $O(n \sqrt{n})$ 。我们还注意到，放宽有效性标准（例如，采用一系列仅局部连接的层）可能对某些领域而言是一种有用的归纳偏差。

在这项工作中，我们探究了 $n = 2$ 的两种因式分解，我们将在下一节对其进行描述，不过需要说明的是，这些技术可以轻松扩展到更高维度。

4.3. 二维因式分解注意力

一种在二维空间中定义因子化注意力模式的自然方法是，让一个注意力头关注之前的 l 个位置，另一个注意力头关注每个 lh 位置，其中 l 是步长，且选择为接近 $\sqrt{n}$ ，我们将这种方法称为跨步注意力。

形式上， $i = \max(0, i - l)$ 和 $A^{(2)} = \{i \cdot (j - i) \bmod l + i\}$ 对应 $A^{(1)} = \{i + 1 \dots i\}$ 。这种模式可以在图 3 (b) 中直观地看到。

如果数据天然具有与步幅相匹配的结构（例如图像或某些类型的音乐），这种公式会很方便。然而，对于没有周期性结构的数据（如文本），我们发现网络可能无法通过步幅模式正确传递信息，因为一个元素的空间坐标不一定与其在未来可能最相关的位置相关联。

在这些情况下，我们转而使用固定的注意力模式（图 3 (c)），其中特定的单元格会总结先前的位置，并将该信息传播到所有后续单元格。

形式上， $\Delta^{(1)} = \{i \cdot (l \cdot i / l) = \{i / l\} \cdot l\}$ ，其中括号表示取整运算，且 $A = \{i \cdot (j / l) = \{i \cdot i \cdot l \bmod l + i + 1 \dots i\}$ ，其中 $i = l - 1$ ， c 是一个超参数。

具体来说，如果步长为 128，且标识符为 $n = 8$ ，那么所有大于 128 的未来位置都可以关注 120-128 的位置，所有大于 256 的位置都可以关注 248-256 的位置。依此类推。

带有 $\sqrt{n}$ 的固定注意力模式极大地限制了网络的表达能力。因为许多表征在

网络仅用于一个块，而少数位置则被所有块使用。相反，我们发现为 $l \in$ 的典型值 $\{128, 256\}$ 选择 $c \in 8, 16, 32$ 效果很好，尽管需要注意的是，与跨步注意力相比，这会使该方法的计算成本增加 c 。

此外，我们发现，在使用多个注意力头时，让它们关注大小为 l 的块内长度为 c 的不同子块，比让它们关注相同的子块更为可取。

在下一节中，我们将介绍如何将因子化注意力整合到稀疏 Transformer 架构中。

· 稀疏 Transformer

在此，我们详细描述了稀疏 Transformer 架构，它是 Transformer (Vaswani 等人, 2017) 的一个修改版本。

5.1. 因式分解注意力头

标准密集注意力只是对式 2 中定义的 attend 函数进行线性变换：

$$\text{attention}(X) = W_n \cdot \text{attend}(X, S) \quad (5)$$

其中， W_n 表示注意力后权重矩阵。整合因式分解自注意力的最简单技术是每个残差块使用一种注意力类型，并按顺序交错排列，或者按作为超参数确定的比例交错排列：

$$\text{attention}(X) = W_n \cdot \text{attend}(X, A^{(r \bmod p)}) \quad (6)$$

这里， r 是当前残差块的索引， p 是分解注意力头的数量。

第二种方法是让单个注意力头关注两个分解注意力头都会关注的像素位置，我们称之为合并注意力头：

$$\text{attention}(X) = W_n \cdot \text{attend}(X, \bigcup_{m=1}^p A^{(m)}) \quad (7)$$

这在计算上稍微更密集一些，但仅相差一个常数因子。第三种方法是使用多头注意力 (Vaswani 等人, 2017)，其中 n_h 注意力乘积是并行计算的，然后沿着特征维度进行拼接：

$$\text{attention}(X) = W_n (\text{attend}(X, A^{(i)})_{i=1, \dots, n_h}) \quad (8)$$

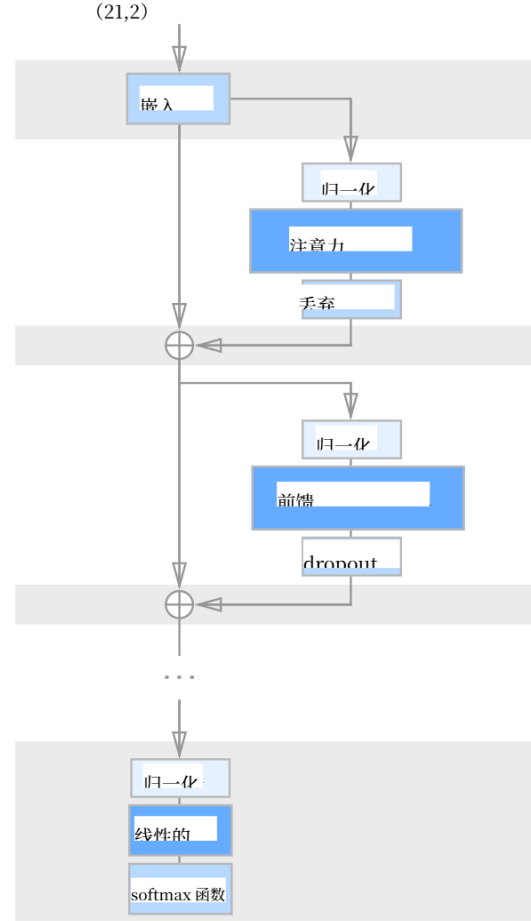


图 4. 稀疏 Transformer 的一个残差块示意图。带阴影的背景表示经过检查点处理 (Chen 等人, 2016) 并存储在 GPU 内存中的张量。其他张量 (包括注意力权重和前馈网络激活值) 会在梯度计算过程中重新计算，从而大幅减少内存使用量。

在这里， A 可以是单独的注意力模式、合并的模式，或者如公式 2 中那样交错的模式。此外， attend 函数内部权重矩阵的维度按 $1/n_h$ 的系数缩减，使得参数数量在 n_h 的不同值之间保持不变。

我们通常发现多个头的效果很好，但对于注意力在计算时间中占主导地位的极长序列，一次只执行一个头并按顺序进行会更值得。

5.2. 扩展到数百层

正如 (Al-Rfou 等人, 2018 年) 所指出的，我们发现带有多层的 Transformer 很难训练。我们没有采用辅助损失，而是采用了以下方法。

架构变化。

首先，我们使用 (He 等人, 2016) 提出的预激活残差块，通过以下方式定义一个包含 N 层的网络：

$$H_0 = \text{embed}(X; W_e)(9)$$

$$H_L = H_{L-1} + \text{resblock}(H_{L-1}) \quad (10)$$

$$u = \text{softmax}(\text{norm}(H_N)W_{att})(11)$$

其中，embed 是我们将在下一节描述的函数， W_{att} 是一个权重 (h) 通过以下方式对注意力块的输入和一个位置前馈网络进行归一化：

$$a(H) = \text{dropout}(\text{attention}(\text{norm}(H)))(12)$$

$$b(H) = \text{dropout}(ff(\text{norm}(H + a(H))))(13)$$

$$\text{resblock}(H) = a(H) + b(H)(14)$$

范数函数表示层归一化 (Ba 等人, 2016)，以及 $ff(x) = W_2 f(W_1 x + b_1) + b_2$ 。我们选择的 f 是高斯误差线性单元 (Hendrycks 和 Gimpel, 2016)， $f(x) = x \odot \text{sigmoid}(1.702 \cdot X)$ ，如 (Radford 等人, 2018) 中所使用的。除非另有说明， W_1 的输出维度是输入维度的 4.0 倍。

注意， H_N 是函数 a 和 b 经过 N 次应用后的总和，因此每个功能块都直接从输出层接收梯度。在式 5 中，我们将 W_1 和 W_2 的初始化按 $\sqrt{\frac{2}{N}}$ 进行缩放，以保持输入嵌入尺度与残差块尺度的比率在 N 的所有值上不变：

5.3. 多样化数据类型的建模

除了输入符号的嵌入外，位置嵌入通常在 Transformer 和其他与位置无关的架构中用于编码数据的空间关系 (Gehring 等人, 2017)、(Parmar 等人, 2018)。

我们发现，使用经过学习的嵌入（这些嵌入要么对数据结构进行了编码，要么对分解后的注意力模式进行了编码）对我们模型的性能至关重要。

我们向每个输入位置添加了 $n_{emb} = d_{data}$ 或 $n_{emb} = d_{att}$ 嵌入，其中 d_{data} 指的是数据的维度数， d_{att} 是分解注意力的维度数。如果 τ_i 是独热编码

序列中独热编码的第 i 个元素，而 $\omega_i^{(j)}$ 表示

x_i 在 j th 维度 ($1 \leq j \leq n_{emb}$) 中的独热编码位置)，则：

$$\text{embed}(X, W_e) = (x_i W_e + \sum_{i=1}^{n_{emb}} \omega_i^{(j)} W_j)_{i=1}^n$$

对于图像，我们使用了数据嵌入，其中 $d_{data} = 3$ 代表每个输入字节的行、列和通道位置。对于文本和音频，我们使用了二维注意力嵌入，其中 $d_{att} = 2$ ，且索引对应于宽度等于步长的矩阵中每个位置的行索引和列索引。

5.4. 通过重新计算注意力权重节省内存

梯度检查点已被证明能有效降低深度神经网络训练的内存需求 (Chen 等人, 2016; Gruslys 等人, 2016)。然而，值得注意的是，当处理长序列时，这种技术对自注意力层尤其有效，因为这些层的内存使用量较高，而计算成本相对较低。

仅使用重新计算，我们就能够在 16,384 的序列长度上训练具有数百层的密集注意力网络，否则这在现代硬件上是不可行的。

在我们的实验中，我们在向后传递期间重新计算注意力和前馈块。为了简化我们的实现，我们没有像在 (Vaswani et al., 2017) 中那样在注意力块中应用 dropout，而是只在每个残差加法的末尾应用它，如图 4 所示。

5.5. 高效的块稀疏注意力内核

3 (b) 和 3 (c) 中的稀疏注意力掩码可以通过从查询、键和值矩阵中切出子块并在块中计算乘积来有效计算。本地窗口上的注意力可以按原样计算，而步幅为 k 的注意力可以通过转置矩阵和计算本地窗口来计算。固定的注意力位置可以在块中聚合和计算。

为了简化实验，我们实现了一组有效执行这些操作的 GPU 内核。softmax 操作融合到单个内核中，并且还使用寄存器来消除多次加载输入数据，允许它以与简单非线性相同的速度运行。注意力矩阵的上三角形永远不会计算，此外，不需要负偏置项 (Vaswani et al., 2017)，并将要执行的操作数量减半。

5.6. 混合精度训练

我们将网络权重存储为单精度浮点数，但像 (Micikevicius 等人, 2017) 中那样，在其他情况下以半精度计算网络激活和梯度。由于在 V100 GPU 上使用了张量核心操作，这加快了我们的训练速度。在梯度计算过程中，我们使用

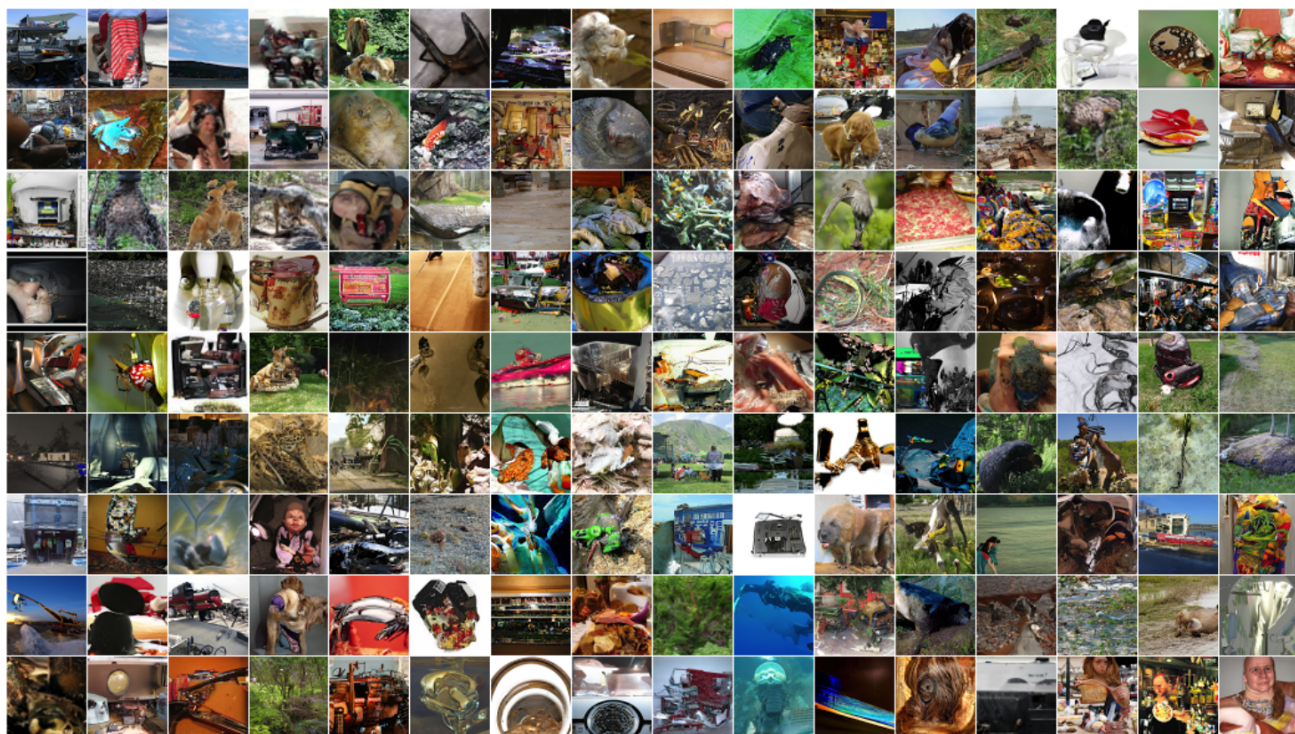


图 5. ImageNet 64x64 的无条件样本，使用未修改的 1.0 softmax 温度生成。我们能够直接从像素中学习长程依赖关系，而无需使用多尺度架构。

我们使用动态损失缩放来减少数值下溢，并在多个 GPU 间进行平均时传递半精度梯度。在采样时，我们将查询和键转换为单精度，因为查询 - 键乘积有时会超过半精度的最大值而导致溢出。

· 训练

我们使用 Adam 优化器，采用 5000 次迭代的线性预热和 1.0 的梯度裁剪，我们发现这两者对模型稳定性都很重要。我们使用 0.01 的权重衰减惩罚。我们按照 (Radford 等人, 2018) 中的余弦衰减来调整学习率。除非另有说明，否则我们在 8 块 V100 GPU 上进行训练。

所有嵌入都具有恒定的维度 d ，通常是 {256, 512, 1024} 中的一个。默认情况下，所有线性变换都采用相同的维度，但前馈网络除外，它会将输入投影到 $4d$ ，除非我们使用“半尺寸”变换，此时投影维度为 $2d$ 。此外，有时我们会将查询和键变换的尺寸减半。

我们从 $N(0, \frac{1}{\sqrt{d}})$ 初始化令牌嵌入 w_{token} ，并从 $N(0, \frac{1}{\sqrt{d}})$ 初始化位置嵌入。在注意力和前馈组件中，所有偏置都被初始

均初始化为 0，所有权重均从 $N(0, \frac{1}{\sqrt{d}})$ 初始化而来，其中 d 为输入维度。输出对数的权重矩阵初始化为 0。

· 实验

我们在包括自然图像、文本和原始音频在内的密度建模任务上对我们的架构进行了实证测试。结果摘要见表 1。我们发现，如表 2 所示，除了运行速度明显快于全注意力机制外，稀疏模式还收敛到了更低的误差。这可能表明我们引入的稀疏模式具有有益的归纳偏置，或者全注意力机制存在潜在的优化问题。

7.1. CIFAR-10

我们在 CIFAR-10 图像上训练跨步稀疏 Transformer，这些图像被表示为 3072 字节的序列。模型有 2 个注意力头、128 层、 $d = 256$ 、半尺寸前馈网络和查询 - 键投影，训练 120 个 epoch，学习率为 0.00035，dropout 率为 0.25，直到验证误差停止下降。

我们使用 48000 个样本进行训练，2000 个样本进行验证，以此来评估我们最佳模型的性能。评估基于

表 1. 我们在密度建模任务中的研究结果摘要。结果以每字节比特数报告，这相当于图像任务中的每维度比特数。M 代表百万参数。

Model	Bits per byte
CIFAR-10	
PixelCNN (Oord et al., 2016)	3.03
PixelCNN++ (Salimans et al., 2017)	2.92
Image Transformer (Parmar et al., 2018)	2.90
PixelSNAIL (Chen et al., 2017)	2.85
Sparse Transformer 59M (strided)	2.80
Enwik8	
Deeper Self-Attention (Al-Rfou et al., 2018)	1.06
Transformer-XL 88M (Dai et al., 2018)	1.03
Transformer-XL 277M (Dai et al., 2018)	0.99
Sparse Transformer 95M (fixed)	0.99
ImageNet 64x64	
PixelCNN (Oord et al., 2016)	3.57
Parallel Multiscale (Reed et al., 2017)	3.7
Glow (Kingma & Dhariwal, 2018)	3.81
SPN 150M (Menick & Kalchbrenner, 2018)	3.52
Sparse Transformer 152M (strided)	3.44
Classical music, 5 seconds at 12 kHz	
Sparse Transformer 152M (strided)	1.97

测试集。该模型实现了每维度 2.80 比特（在种子 1、2、3 上为 2.798 ± 0.004 ），而之前的最先进水平为 2.85（Chen 等人，2017）。我们还在表 2 中比较了不同注意力模式的性能。跨步注意力在最短时间内达到了最低误差，超过了密集注意力 2.82 比特每维度的误差。

7.2. 文本

为了在没有明显二维结构的数据集上评估稀疏 Transformer，我们在 EnWik8 数据集上训练了模型。该数据集包含维基百科的前 108 字节内容，其周期结构具有很大的可变性。我们训练时使用的上下文长度为 12,288，这比以往的方法更长。

我们在首批 9000 万个标记上进行了训练，并预留了最后 1000 万个标记用于验证和测试。我们使用了 30 层固定的稀疏 Transformer，包含 8 个注意力头、 $d = 512$ ，以及 0.40 的 dropout 率。我们训练了 80 个 epoch，直到验证损失不再下降。我们使用了 128 的步长、 $n = 32$ ，并合并了因式分解的注意力头。

我们最好的模型达到了每维度 0.99 比特（在种子 1、2、3 上为 0.992 ± 0.001 ），超过了同等规模 Transformer-XL 的 1.03 这一最先进水平（Dai 等人，2018 年），并与训练数据量超过两倍的模型所达到的 0.99 持平。

表 2. 在我们可以对两者进行比较的数据集上，稀疏模式不仅显示出更快的速度，还表现出更好的损失值，这可能表明我们所学习的模式中存在有用的归纳偏差，或者全注意力存在潜在的优化问题。

Model	Bits per byte	Time/Iter
Enwik8 (12,288 context)		
Dense Attention	1.00	1.31
Sparse Transformer (Fixed)	0.99	0.55
Sparse Transformer (Strided)	1.13	0.35
CIFAR-10 (3,072 context)		
Dense Attention	2.82	0.54
Sparse Transformer (Fixed)	2.85	0.47
Sparse Transformer (Strided)	2.80	0.38

表 3. 我们观察到随着上下文变长，Enwik8 的压缩率有所提高，这表明稀疏 Transformer 能够有效地整合长期依赖关系。

Minimum context length during evaluation	Bits per byte
6,144 tokens	0.9952
9,216 tokens	0.9936
10,752 tokens	0.9932
11,904 tokens	0.9930
12,096 tokens	0.9922
12,160 tokens	0.9908

参数数量。跨步注意力在该数据集上表现不佳，而固定模式能够恢复并超越密集注意力的性能，如表 2 所示。

此外，在测试集评估过程中，我们通过减少并行评估的标记数量，修改了网络可使用的最小上下文长度。我们发现，随着使用标记数量的增加，性能呈单调上升趋势，最多使用 12,160 个标记（训练时使用了 12,288 个标记）（见表 3），这表明该网络能有效整合长期依赖关系。

7.3. ImageNet 64x64

为了测试该模型学习长程依赖关系以及在大规模数据集上的扩展能力，我们在（Oord 等人，2016）发布的下采样 ImageNet 版本上进行训练，并在验证集上进行评估。我们使用了一个 48 层带步长的稀疏 Transformer，它有 16 个注意力头和 $d = 512$ ，总参数为 1.52 亿。我们采用的步长为 128，dropout 为 0.01，训练了 70 个 epoch，这在 64 块 V100 GPU 上花费了 7 天时间。

我们的模型实现了每维度 3.44 比特的损失（1 次运行中为 3.437），相比之下，之前的模型为 3.52（Menick & Kalchbrenner，2018）。

此外，我们从该模型以及一个经过两倍层数（总参数为 3 亿）训练的模型中，在未修改的 softmax 温度 1.0 下生成了无条件样本（图 5）。我们在此包含了 3 亿参数模型的样本。通过视觉评估，我们未发现稀疏模式带来的伪影，并且在大多数图像中看到了长期结构的迹象。

7.4. 来自原始音频的古典音乐

为了测试稀疏 Transformer 在超长语境下的扩展能力，我们在（Dieleman 等人，2018）发布的古典音乐数据集上训练了模型。由于该数据集的处理细节不可得，我们没有与其他研究进行直接比较，而是研究了随着语境规模的增大，我们能够训练的稀疏 Transformer 的规模。对于每个序列长度，我们尝试训练最大的模型，该模型无需模型并行就能完全适配 16GB 的 V100 加速器。

总体而言，我们发现将序列长度增加 4 倍需要将模型容量减少约 $4\sqrt{4} = 8$ 。因此，我们发现可以在超过 100 万个时间步长的序列上使用因式分解自注意力，尽管参数极少（300 万个）。

有长度为 65536 的序列样本可供使用，这些样本对应于 12kHz 下约 5 秒的生成音频。这些样本清楚地展示了在采样期间的全局连贯性，并呈现出多种演奏风格和音调，从有节奏的演奏转换为有力的演奏。要收听样本，请访问 <https://openai.com/blog/sparse-transformer>。由于模型容量降低，对于更长的序列长度，样本质量会迅速下降。

表 4. 跨步稀疏 Transformer 在经典音频数据集（以 12 kHz 进行 μ 律编码）上的性能随序列长度和模型大小的变化。

Sequence length	Parameters	Bits per byte
65,536	152M	1.97
262,144	25M	2.17
1,048,576	3M	2.99

· 结论

我们引入了稀疏 Transformer，并表明它们在长序列的密度建模上表现出与标准 Transformer 相当或更优的性能，同时所需的运算量显著减少。这种性能在图像和文本领域达到了最先进水平，并且易于适配原始音频。该模型展示了对长期上下文的利用能力，并能生成具有全局连贯性的样本。

· 致谢

我们要感谢阿希什·瓦斯瓦尼在项目初期的富有洞察力的讨论。我们还要感谢约书亚·迈尔和马克·陈的有益讨论，以及约翰内斯·奥特巴赫、普拉富拉·达里瓦尔和大卫·卢安对本文初稿的反馈。

参考文献

Al-Rfou, R., Choe, D., Constant, N., Guo, M. 和 Jones, L. 基于更深层自注意力的字符级语言建模。arXiv 预印本 arXiv:1808.04444. 2018。

巴, J. L., 基罗斯, J. R. 和辛顿, G. E. 层归一化。arXiv 预印本 arXiv:1607.06450. 2016。

Britz, D., Guan, M. Y. 和 Luong, M.-T. 利用固定大小记忆表示的高效注意力机制。arXiv 预印本 arXiv:1707.00110. 2017。

陈涛、徐博、张聪和塞巴斯蒂安·格斯汀。《以亚线性内存成本训练深度网络》。arXiv 预印本 arXiv:1604.06174. 2016 年。

陈希、米什拉 (N.)、罗汉尼贾德 (M.) 和阿比尔 (P.)。《PixelSnail: 一种改进的自回归生成模型》。arXiv 预印本 arXiv:1712.09763. 2017 年。

邱彩云 (C.-C. Chiu) 和拉斐尔 (C. Raffel)。单调分块注意力机制。arXiv 预印本 arXiv:1712.05382. 2017 年。

戴子航、杨子越、杨毅、W. W. 科恩、J. 卡博内尔、Q. V. 乐和 R. 萨拉赫丁诺夫。Transformer-xl: 具有更长期依赖关系的语言建模。2018 年。

Dieleman, S., van den Oord, A. 和 Simonyan, K. 现实音乐生成的挑战：大规模建模原始音频。见《神经信息处理系统进展》，第 8000-8010 页。2018 年。

盖林 (J.)、奥利 (M.)、格兰杰 (D.)、亚拉茨 (D.) 和多芬 (Y. N.)。卷积序列到序列学习。arXiv 预印本 arXiv:1705.03122. 2017。

Gruslys, A., Munos, R., Danihelka, I., Lanctot, M. 和 Graves, A.。时间反向传播的内存高效方法。见《神经信息处理系统进展》，第 4125-4133 页。2016 年。

何凯明、张祥雨、任少卿、孙剑。深度残差网络中的恒等映射。arXiv 预印本 arXiv:1603.05027, 2016。

亨德里克斯 (D.) 和金佩尔 (K.)。利用高斯误差线性单元桥接非线性和随机正则化器。arXiv 预印本 arXiv:1606.08415. 2016。

黄, C.-Z. A.、瓦斯瓦尼, A.、乌兹科雷特, J.、沙泽尔, N.、霍桑, C.、戴, A. M.、霍夫曼, M. D. 和埃克, D. 一种改进的 Transformer 相对自注意力机制及其在音乐生成中的应用。arXiv 预印本 arXiv:1809.04281, 2018。

Jozefowicz, R.、Vinyals, O.、Schuster, M.、Shazeer, N. 和 Wu, Y. 探索语言建模的极限。arXiv 预印本 arXiv:1602.02410. 2016。

Kingma, D. P. 与 Dhariwal, P. 《Glow: 具有可逆 1×1 卷积的生成流》, 收录于《神经信息处理系统进展》第 10236–10245 页。2018 年。

Koutnik, J., Greff, K., Gomez, F., 和 Schmidhuber, J. 一种发条循环神经网络。arXiv 预印本 arXiv:1402.3511, 2014。

刘, P. J., 萨利赫, M., 波特, E., 古德里奇, B., 塞帕西, R., 凯泽, L., 以及沙泽尔, N. 通过总结长序列生成维基百科。arXiv 预印本 arXiv:1801.10198, 2018。

Mehri, S., Kumar, K., Gulrajani, I., Kumar, R., Jain, S., Sotelo, J., Courville, A. 和 Bengio, Y. Samplernn: 一种无条件端到端神经音频生成模型。arXiv 预印本 arXiv:1612.07837, 2016。

Menick, J. 和 Kalchbrenner, N. 利用子尺度像素网络和多维上采样生成高保真图像。arXiv 预印本 arXiv:1812.01608. 2018。

米西凯维丘斯 (P.)、纳兰 (S.)、阿尔本 (J.)、迪亚莫斯 (G.)、埃尔森 (E.)、加西亚 (D.)、金斯伯格 (B.)、休斯顿 (M.)、库恰耶夫 (O.)、文卡泰什 (G.) 等。混合精度训练。arXiv 预印本 arXiv:1710.03740, 2017。

Oord, A. v. d., Kalchbrenner, N. 和 Kavukcuoglu, K. 像素循环神经网络。arXiv 预印本 arXiv:1601.06759. 2016。

帕马尔 (N.)、瓦斯瓦尼 (A.)、乌兹科雷特 (J.)、凯泽 (L.)、沙泽尔 (N.) 和库 (A.)。《图像转换器》。arXiv 预印本 arXiv:1802.05751. 2018 年。

Radford, A., Narasimhan, K., Salimans, T. 和 Sutskever, I. 借助生成式预训练提升语言理解能力。网址: <https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language-understanding-paper.pdf>, 2018。

里德 (S.)、奥德 (A. v. d.)、卡尔克布伦纳 (N.)、科尔梅纳雷霍 (S. G.)、王 (Z.)、别洛夫 (D.) 和德弗雷塔斯 (N.)。并行多尺度自回归密度估计。arXiv 预印本 arXiv:1703.03664, 2017。

Salimans, T., Karpathy, A., Chen, X. 和 Kingma, D. P. Pixelcnn++: 通过离散逻辑混合似然及其他修改改进 Pixelcnn。arXiv 预印本 arXiv:1701.05517. 2017。

Sukhbaatar, S., Weston, J., Fergus, R. 等人。端到端记忆网络。见《神经信息处理系统进展》, 第 2440–2448 页。2015 年。

Van Den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. 和 Kavukcuoglu, K. WaveNet: 一种原始音频生成模型。CoRR abs/1609.03499, 2016。

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. 和 Polosukhin, I. 注意力就是全部。收录于《神经信息处理系统进展》, 第 5998–6008 页, 2017 年。